And then we finalise the changes and check noob's privileges, as shown below:

```
FLUSH PRIVILEGES;
SHOW grants FOR 'noob'@'localhost;
```

These are a few baby steps we can take to ensure minimal security in our database. An exhaustive resource for securing MySQL can be found at *https://www.symantec.com/connect/articles/securing-mysql-step-step*.

Securing PHP

Do use PHP 7+ because unless you are working on a huge monolithic legacy codebase from 2009, there's no reason to still use PHP 5 or earlier. The number of improvements in terms of security and performance in PHP 7 is enormous, and it has been getting better with every update. PHP 7.4 and PHP 8 are currently in development.

The best thing you can do to improve security in PHP is to update it. PHP 7+ addressed most of the problems that PHP critics had pointed out earlier. PHP 7.3.2 is already out and it is recommended that you stay with the updated stable version. Although migration may take time, since there are a lot of backwards-incompatible changes, it is worth all the developer's time.

Enabling strict types: One of the most basic things you can do to make your life easier and write better code is enabling strict types in PHP. This will take you a long way towards the goal of making your application safe from bugs.

To make sure you don't end up passing a string to an integer argument, start all your PHP files as follows:

```
<?php declare(strict_types=1);
```

And now you can define a function as shown below:

```
function getUser(int id) : User
{
    ...
}
```

With modern IDEs like PHPStorm, you will get a warning directly if you pass an argument with a wrong type while calling the function.

Using PDO and prepared statements: PDO (PHP Data Objects) is the object-oriented way to talk to a database from PHP and should be the only way to do so. It is a database abstraction layer (DAL), which abstracts away the low-level details for connecting to the database in a safe manner. *mysql_** functions have long been deprecated and removed in PHP 7.

PDO takes away the dangers of the popular MySQL injections using prepared statements and bind parameters in its queries. Its reusable API allows you to connect to any database without changing much code.

In 2019, on a modular Web application, you probably would want to consider ORMs (object-relational mappers) with query builders like Doctrine for adding further abstraction.

Validating user input and sanitising output: Any security engineer will tell you to never trust user inputs. As a rule of thumb, validate every input you get from the user and store it into the database only after every test passes, or fails completely.

There is no one standard way to achieve this; although a few open source validation libraries do exist, it is generally not a good option to use monolithic validator classes to validate everything, since they usually end up doing too many things and violating the single responsibility principle.

One way is to use value objects and pass them as arguments to your entity classes in the model layer. The type checking should take care of the correctness of the data and throw an exception on failure.

Here is an example. Create an immutable email value object class to hold the email data, as follows:

```
class Email
{
    private $email;

    public function __construct(string $email)
    {
      if ($this->isValid($email))
          $this->email = $email;
      else
          throw new \InvalidArgumentException();
    }
}
```

Then pass it as an argument to your model class that stores user data, as follows:

```
class User
{
    public function __construct(…, Email $email)
    {
      // Do something with email
    }
```

This way we can simplify the process of validating data using value objects. A repository that I am currently working on to provide a set of predefined common value objects can be found at *https://github.com/2DSharp/Phypes*.

Sanitisation, on the other hand, should not be done while inserting data into the database. It should rather be done while displaying data from the database. Sanitising can protect against XSS attacks by turning HTML special characters to appear as regular text. For example, PHP's *filter_var()* functions *htmlspecialchars()* can be used to sanitise output data.

Enable 'display errors' only during development and not production: A common mistake I have seen a lot of